

Django 教學 05

模型關聯與Migration

第 8～9 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [04_Model與欄位設計](#)。

- **第 8 章：模型關聯**
- **第 9 章：Meta、驗證與資料結構演進**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 8 章

模型關聯

畫出關係的兩個方向，驗證關聯查詢及刪除行為。

本章的操作環境與成果

catalog 為完整帶做範例；board／marketplace 節錄需切到對應專案。Profile／Tag 等為教學延伸。

完成成果： 畫出關係的兩個方向，驗證關聯查詢及刪除行為。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。進階頁可回查，但所有基本驗收需完成。

8-1 關聯先看兩個方向的數量

需求	正向	反向	選擇
留言與作者	一則留言至多一位作者	一位作者多則留言	ForeignKey
帳號與個人檔案	一份檔案一個帳號	一個帳號至多一份檔案	OneToOneField
商品與標籤	一件商品多個標籤	一個標籤多件商品	ManyToManyField

「一對多」與「多對一」是同一條關係的兩個方向。
外鍵放在「多」的一方；是否允許沒有對象，再由 null 等設定決定。

8-2 LearnBoard：留言指向作者

抽入 LearnBoard 第 9 章 | Message，節錄

```
from django.conf import settings
from django.db import models

class Message(models.Model):
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True, related_name="messages",
    )
```

一則留言可沒有作者；刪除帳號後，留言保留、author 變成 NULL。
關聯 User 用 AUTH_USER_MODEL，避免綁死 Django 的預設 User。

8-3 外鍵值與關聯物件

```
message.author_id      # 儲存的主鍵值，或 None
message.author         # User instance，或 None
user.messages.all()    # 反向 RelatedManager 產生 QuerySet
user.messages.filter(content__icontains="Django")
```

讀 `author_id` 通常不需另外載入 `User`；讀 `author` 可能查資料庫。
`related_name` 決定反向入口名稱，不會把資料庫欄位改名為 `messages`。

沒有設定 `related_name` 時，預設反向存取常是 `message_set`；
反向查詢名稱通常是 `message`。本課採明確命名，降低混淆。

8-4 on_delete 與刪除方向

策略	刪除被參照的對象時	專案例子
CASCADE	連帶刪除參照列	刪 Order → OrderItem
PROTECT	有參照就拒絕刪除	Category ← Product
SET_NULL	保留列並清空 FK	User ← Message

SET_NULL 需搭配 `null=True`。刪留言不會反過來刪作者。

商品同時被訂單保護時，即使 seller 的 FK 用 CASCADE，也可能刪不成。

`on_delete` 是 Django 刪除收集行為，不是宣告 SQL 的 ON DELETE 子句。

8-5 一對一：帳號的額外資料

教學延伸 | 新 app practice 的 models.py，兩個專案尚未加入

```
from django.conf import settings
from django.db import models

class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
```

Profile 有自己的自動 id；user_id 另有唯一性。

一個帳號至多一份 Profile；不代表建立帳號時會自動建立 Profile。

8-6 一對一的正反向都是單一物件

教學延伸 | 完成 Profile migration 後

```
profile, created = Profile.objects.get_or_create(user=user)
profile.user           # User instance
user.profile          # Profile instance, 不是 .all()
```

沒有 Profile 時，讀反向關聯會拋 RelatedObjectDoesNotExist，不是回空 QuerySet，也不是自動回 None。

ForeignKey(unique=True) 在資料唯一性上類似一對一，但 OneToOneField 的反向 API 更直接表達「單一物件」。

8-7 多對多：商品可以貼多個標籤

教學延伸 | 建立 practice app，指向現有 Product

```
class Tag(models.Model):  
    name = models.CharField(max_length=30, unique=True)  
    products = models.ManyToManyField(  
        "marketplace.Product", related_name="tags", blank=True,  
    )
```

關聯只需宣告在其中一邊；另一邊會有反向入口。

Django 建立中介表（專門保存兩筆資料之間的關係），保存 tag_id 與 product_id，預設防止同一組重複。

blank=True 允許不選標籤；ManyToMany 的 null 設定沒有意義。

8-8 多對多的資料其實在第三張表

商品表 id／name	中介表 product_id／tag_id	標籤表 id／name
10／鍵盤	10／1	1／二手
11／滑鼠	10／2	2／可議價
—	11／1	—

鍵盤有兩個標籤；「二手」同時貼在鍵盤與滑鼠。

不要把標籤存成 "二手,可議價"，否則難以可靠地關聯、查詢與去重。

想一想：移除 10／1 這筆關係，需要刪掉「二手」標籤本身嗎？

8-9 add、remove、set、clear

教學延伸 | Product 與 Tag 都必須已儲存

```
tag, _ = Tag.objects.get_or_create(name="二手")
product.tags.add(tag)
product.tags.all()
tag.products.all()
product.tags.remove(tag)
product.tags.set([tag])
product.tags.clear()
```

add／remove 變更關係；set 以指定集合取代；clear 移除全部關係。
這些方法會寫入中介表，不用再呼叫 product.save()。
remove／clear 不會刪除商品或標籤本身。

8-10 進階 | through：關係本身還有資料

教學延伸 | 若「貼標籤時間」也是資料，改用明確中介模型

```
class ProductTag(models.Model):
    product = models.ForeignKey(
        "marketplace.Product", on_delete=models.CASCADE)
    tag = models.ForeignKey("Tag", on_delete=models.CASCADE)
    added_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [models.UniqueConstraint(
            fields=["product", "tag"], name="practice_product_tag",
        )]
```

在 Tag.products 宣告 `through="ProductTag"`；新增關係亦可直接建立 ProductTag。
這是另一版 schema，別直接改已有資料的 M2M 而忽略資料遷移。

8-11 CartItem 與 OrderItem 的建模意義

抽入 LearnMart 商城資料規則

- CartItem 連接 User 與 Product，還保存 quantity。
- OrderItem 連接 Order 與 Product，保存商品名與單價快照。
- 兩者都有中介資料的意義，但目前沒有宣告為 ManyToMany 的 through。
- 舊訂單不能只讀目前 Product.price，否則改價會改變歷史呈現。

關聯不是只畫線；要問「這條關係還要保存什麼歷史事實？」

8-12 訂單快照：保留購買當下的資料

OrderItem 同時保留 relation 與購買當下資料：

```
product = models.ForeignKey(Product, on_delete=models.PROTECT, ...)
product_name = models.CharField(max_length=150)
unit_price = models.DecimalField(max_digits=10, decimal_places=0)
quantity = models.PositiveIntegerField()
```

商品日後改名或漲價，舊訂單仍顯示購買當時名稱與單價。

本章先理解 schema 決策；第 19 章再完成快照、扣庫存與結帳交易。

8-13 關聯練習（20 分鐘）

畫出 User、Profile、Message、Product、Tag 的關係。

每條線標出：數量、FK 所在一方、可否為空、刪除策略與反向名稱。

驗收題：

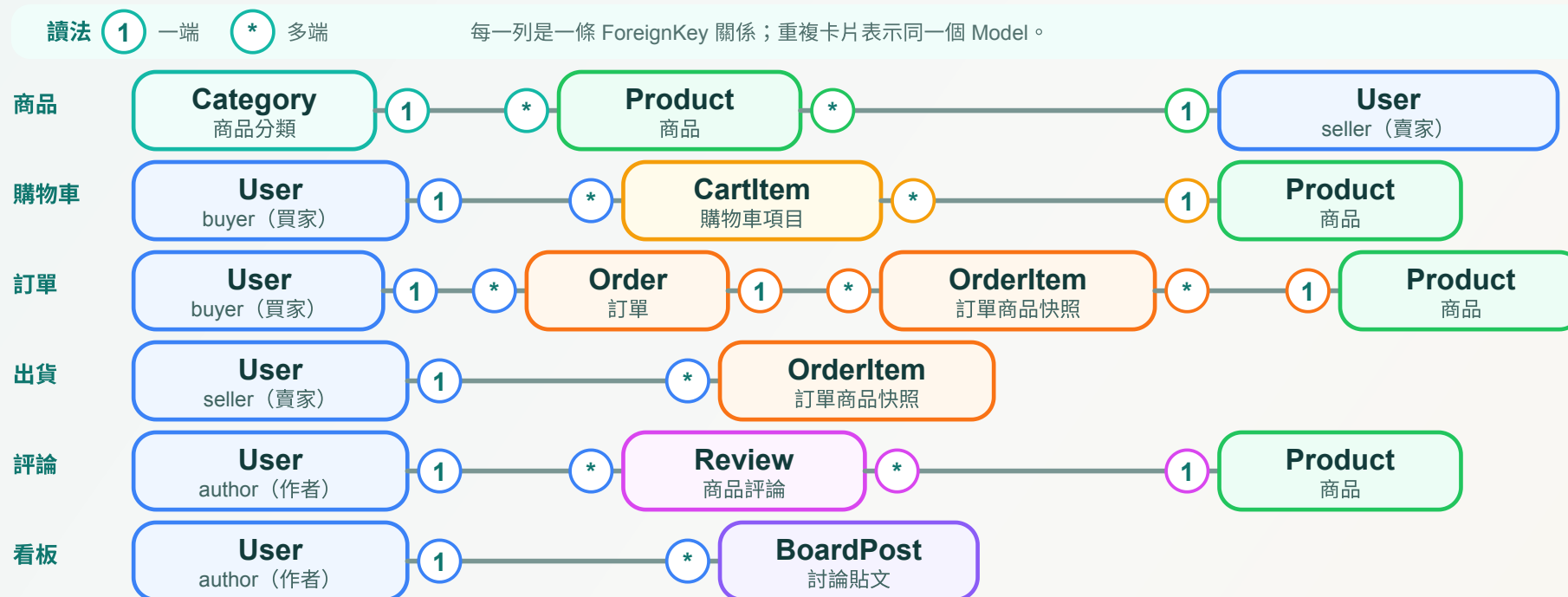
1. 讀 `user.profile` 為何不用 `all()`？不存在時會怎樣？
2. 同一個 `user` 能有兩則留言嗎？兩份 `Profile` 呢？
3. 移除一個商品的標籤，為何不應刪除 `Tag`？
4. 商品單價快照要放在 `Product`，還是 `OrderItem`？

8-14 從一個 Message 到商城的關聯圖

觀察角度	LearnBoard	LearnMart
最小資料核心	Message 的作者、內容、建立時間	Product 的分類、賣家、價格、庫存與圖片
關聯複雜度	一則留言對應一位作者	商品還會連到購物車、訂單明細與評價
專案特有需求	作者顯示與留言搜尋	自訂 User、圖片處理、分類篩選與購買歷史
為何要先學小專案	能清楚追一筆資料如何顯示	再把同一條資料流延伸到多個關聯與規則

不要把兩個 model 硬湊成同一份 class。先找出各自的資料責任，才能正確判斷要用 `select_related`、`prefetch_related`、`constraint` 或 `migration` 的時機。

8-15 專案導覽：先看資料之間的關係



1 ——— * 表示一對多。User 在不同關係中扮演 buyer、seller、author。

先找出商品連到哪些資料；本章已學關聯與刪除規則，請逐條說明圖中的關係。

8-16 一對一與多對多實作路線

在 LearnMart 的練習分支；既有 Product 不需增加欄位

```
uv run python manage.py startapp practice
```

1. 加入 INSTALLED_APPS；在 practice/models.py 放第 8 章 Profile、Tag。
2. 建立並套用 practice migration；先採自動中介表版本。
3. shell 匯入 Profile、Tag、Product，取得一位 user 與一件商品。
4. 建立 Profile，新增兩個 Tag，用 add/remove/set/clear 觀察關係。

驗收：user.profile 是單一物件；product.tags.all() 是集合。

第 8 章實作與離堂檢核

任務： 畫出關係的兩個方向，驗證關聯查詢及刪除行為。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 01 原第 4 章](#)；[LearnMart 01 原第 4 章](#)。手冊保留原章號，對照表列出本課位置。

第 9 章

Meta、驗證與資料結構演進

修改模型，檢查 migration operations，確認套用狀態與資料規則。

本章的操作環境與成果

catalog 為完整帶做範例；board／marketplace 節錄需切到對應專案。Profile／Tag 等為教學延伸。

完成成果： 修改模型，檢查 migration operations，確認套用狀態與資料規則。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。進階頁可回查，但所有基本驗收需完成。

9-1 Model、表單與 Admin 的設定位置

位置	管理的事	例子
Model 欄位	單欄位型別與規則	max_length、blank
Model.Meta	模型整體設定	ordering、constraints
ModelForm.Meta	表單對應模型與可編輯欄位	model、fields
ModelAdmin	後台列表與編輯介面	list_display、search_fields

Model.Meta 不放 fields 白名單；ModelForm.Meta 不負責資料庫索引。
它們名字相似，但屬於不同 class，也由不同元件解讀。

9-2 ordering 與顯示名稱

現有 Category.Meta

```
class Meta:
    verbose_name = "商品分類"
    verbose_name_plural = "商品分類"
    ordering = ["name"]
```

ordering 是預設查詢排序；`order_by("-name")` 可覆蓋它。

排序欄位值若相同，需額外唯一鍵才能確保穩定次序。

教學延伸：商品可用 `["-created_at", "-pk"]`，時間相同時再用 pk 排序。

中文常把單複數名稱設成一樣；這不會改變資料表名稱。

9-3 進階 | db_table 與 indexes

教學延伸 | Product.Meta 的設計候選，不是現有設定

```
class Meta:
    db_table = "marketplace_product"
    indexes = [models.Index(
        fields=["is_active", "-created_at"],
        name="product_active_created_idx",
    )]
```

預設表名已是 app_label+model 名，通常不用另設 db_table。
索引協助特定查詢，但增加儲存與寫入成本；先量測，不要每欄都加。
索引不保證值唯一；唯一性另用 unique 或 UniqueConstraint。

9-4 完整的 Meta.constraints

現有 LearnMart | CartItem，關聯欄位省略

```
class CartItem(models.Model):  
    # user、product、quantity 等欄位在此宣告  
  
    class Meta:  
        constraints = [  
            models.UniqueConstraint(  
                fields=["user", "product"],  
                name="unique_cart_product",  
            ),  
        ]
```

name 是約束名稱；fields 是一起判斷的欄位組合。

新增前先清理既有重複資料；建立 migration 不會替你決定保留哪一筆。

9-5 Review 評價唯一約束

目前 LearnMart 節錄 | `Review.Meta.constraints`

```
models.UniqueConstraint(  
    fields=["product", "author"],  
    name="one_review_per_product",  
)
```

同一 author/product 只保留一筆 review；目前 workflow 使用 `update_or_create()` 更新內容。

這個 database uniqueness 與 rating 1-5 validator 是不同規則、不同防線。

9-6 CheckConstraint：範圍與跨欄位規則

教學延伸 | 可加入 `CartItem.Meta.constraints`

```
models.CheckConstraint(  
    condition=models.Q(quantity__gte=1),  
    name="cart_quantity_at_least_one",  
)
```

- validator 幫使用者得到輸入錯誤訊息。
- CheckConstraint 讓資料庫拒絕不符合條件的寫入。
- 這與 user+product 的「不可重複」是兩條不同規則。
- nullable 欄位還需明確思考 NULL 的邏輯與資料庫差異。

目前 CartItem 有 MinValueValidator(1)，尚未加入本頁的 CHECK。

9-7 驗證不是只有一個入口

入口	會發生什麼？
Admin／ModelForm.is_valid()	做表單與相應的模型驗證；排除欄位有邊界
instance.full_clean()	欄位、clean、唯一性、約束驗證
instance.save()／objects.create()	不自動呼叫完整 full_clean
QuerySet.update()	直接 SQL 更新，不逐筆 save
資料庫約束	最後擋住受約束的非法寫入

驗證通過到寫入之間仍可能有人先寫入；unique 的競態需處理 IntegrityError。
不能只做 exists() 檢查就宣稱不會重複。

9-8 Model、migration 與資料表

```
修改 models.py
    ↓ makemigrations
產生 migration operations
    ↓ migrate
更新資料庫 schema
```

- makemigrations 比較模型與 migration 狀態，不是比較實際 DB 的所有差異。
- migrate 套用版本紀錄；不是每次把所有表重建。
- 改 Meta 也可能產生 migration；不一定都對應實際 SQL 變更。
- Admin 的列表設定不屬於 schema，通常不需 migration。

9-9 取得專案與修改模型的操作差異

clone／取得既有 repository

```
uv run python manage.py migrate
```

Migration 已存在；把它套用到本機 database。

你真的改了 model

```
uv run python manage.py makemigrations  
# 先閱讀產生的 migration  
uv run python manage.py migrate
```

`showmigrations` 是檢查套用狀態；不是每次必須的第三個 schema-changing 步驟。

9-10 migration 訊息與狀態檢查

No changes detected

通常表示 Django 比較 model state 與 migration state 後沒有新差異；在完成版 LearnMart 上這可能是正常結果。

Your models ... have changes that are not yet reflected in a migration

表示 model 與 migration 不一致。

驗證指令：

```
uv run python manage.py makemigrations --check
```

成功退出表示沒有未建立的 model migration。

9-11 新增欄位的操作與證據

教學延伸 | 在 LearnMart 練習分支新增 is_featured

```
is_featured = models.BooleanField("精選", default=False)
```

先修改 models.py，再用下一頁命令產生並閱讀 migration。
現有資料也必須有合理預設值；本例用 False 表示尚未設為精選。

9-12 產生、檢查並套用 migration

```
uv run python manage.py makemigrations marketplace
uv run python manage.py sqlmigrate marketplace 0002
uv run python manage.py migrate
uv run python manage.py makemigrations --check
```

0002 請換成實際產生的 migration 編號；先讀 operations 與 SQL 再套用。
驗收：現有商品有合理值，Admin 可顯示，沒有遺漏的 migration。

9-13 既有資料讓變更更困難

新增必填欄位時，舊資料要填什麼？新增 unique 時，舊值是否重複？

1. 先定義舊資料如何補值，不要隨意接受同一個假值。
2. 必要時先允許空值，另做資料遷移補齊。
3. 確認符合規則，再加 NOT NULL／unique／constraint。
4. 評估鎖表、備份與回復方式，接續第 26 章的部署與復原。

配套：保留 LearnBoard 的 0002_message_author.py，觀察後加作者欄位。
不要刪掉既有 migration 來掩蓋問題。

9-14 Migration 三步驟不要背成一團

models.py 改變



makemigrations 產生「變更計畫檔」



migrate 把 migration 套到 database

常用：

```
uv run python manage.py makemigrations
```

```
uv run python manage.py migrate
```

9-15 makemigrations --check 是什麼？

```
uv run python manage.py makemigrations --check
```

用途：確認 `models.py` 沒有尚未建立 migration 的變更。

它不是「執行 migration」。

9-16 看 migration，不要只相信它

```
uv run python manage.py showmigrations  
uv run python manage.py migrate --plan
```

某 app 的 SQL：

```
uv run python manage.py sqlmigrate board 0002
```

把「Python migration operation」連到「資料庫 schema 真的會怎麼改」。

9-17 修改 model 後的固定節奏

```
uv run python manage.py makemigrations  
uv run python manage.py migrate  
uv run python manage.py check  
uv run python manage.py test
```

不要只 reload browser。

Model schema 改變需要 database 跟著改。

第 9 章實作與離堂檢核

任務： 修改模型，檢查 migration operations，確認套用狀態與資料規則。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： LearnBoard 02 原第 3 章；LearnMart 01 原第 4 章。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[06_ORM與Admin](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。